

Параллельные методы матричного умножения

- Последовательный алгоритм
- Базовый параллельный алгоритм
- Алгоритм умножения матриц, основанный на ленточном разделении данных
- Блочный алгоритм умножения матриц
- Блочный алгоритм, эффективно использующий кэш-память

Постановка задачи

- Умножение матрицы A размера $m \times n$ и матрицы B размера $n \times l$ приводит к получению матрицы C размера $m \times l$, каждый элемент которой определяется в соответствии с выражением:

$$c_{ij} = \sum_{k=0}^{n-1} a_{ik} \cdot b_{kj}, 0 \leq i < m, 0 \leq j < l.$$

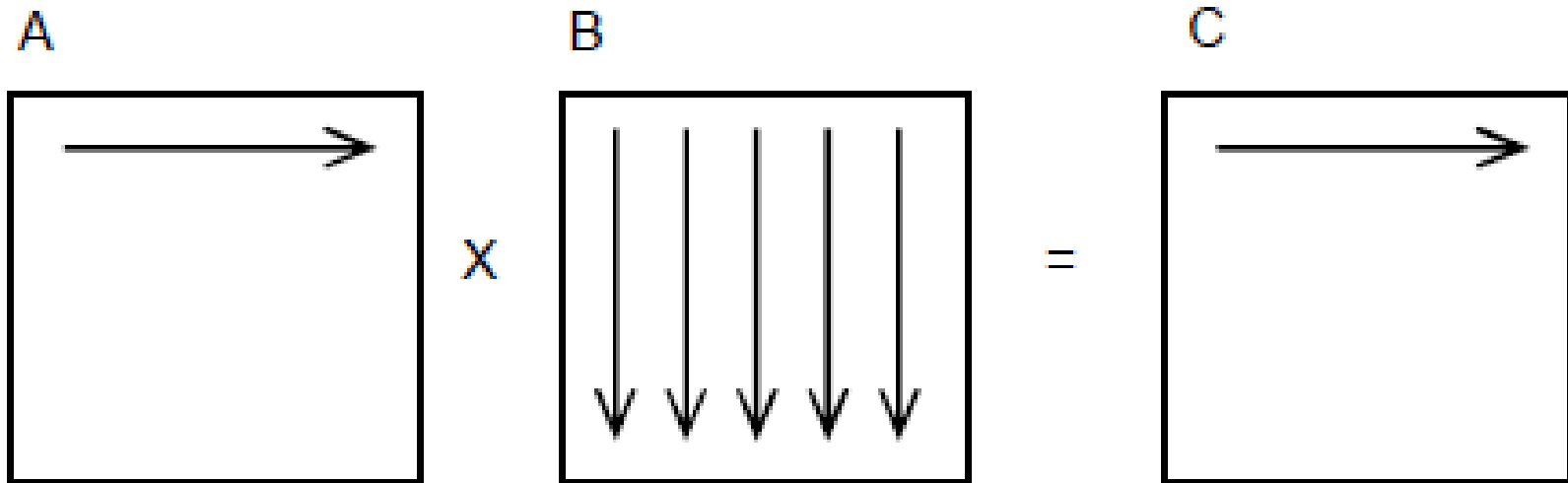
- Каждый элемент результирующей матрицы C есть скалярное произведение соответствующих строки матрицы A и столбца матрицы B :

$$c_{ij} = (a_i, b_j^T), \quad a_i = (a_{i0}, a_{i1}, \dots, a_{i,n-1}), \quad b_j^T = (b_{0j}, b_{1j}, \dots, b_{n-1j})^T$$

Последовательный алгоритм

```
// Последовательный алгоритм умножения матриц
double MatrixA[Size][Size];
double MatrixB[Size][Size];
double MatrixC[Size][Size];
int i,j,k;
...
for (i=0; i<Size; i++){
    for (j=0; j<Size; j++){
        MatrixC[i][j] = 0;
        for (k=0; k<Size; k++){
            MatrixC[i][j] = MatrixC[i][j] + MatrixA[i][k]*MatrixB[k][j];
        }
    }
}
```

- Этот алгоритм является итеративным и ориентирован на последовательное вычисление строк матрицы C .
- При выполнении одной итерации внешнего цикла (цикла по переменной i) вычисляется одна строка результирующей матрицы



Анализ эффективности

- Оценка времени выполнения последовательного алгоритма умножения матриц может быть представлена следующим образом:

$$T_1 = n^2(2n - 1) \cdot \tau + 64 \cdot (2n^3 + n^2) / \beta$$

где β - пропускная способность канала доступа к оперативной памяти (константа 64 введена для учета факта, что в случае кэш-промаха из ОП читается кэш-строка размером 64 байт).

Анализ эффективности

- Если помимо пропускной способности учесть латентность памяти, модель приобретет следующий вид:

$$T_1 = n^2(2n - 1) \cdot \tau + (2n^3 + n^2)(\alpha + 64 / \beta)$$

где α есть латентность оперативной памяти.

- Обе предыдущие модели являются моделями на худший случай и дают сильно завышенную оценку времени выполнения алгоритма, так как доступ к оперативной памяти происходит не при каждом обращении к элементу

Анализ эффективности

- Введем в разработанную модель величину γ , $0 \leq \gamma \leq 1$, для задания частоты возникновения кэш промахов. Тогда оценка времени выполнения алгоритма матричного умножения принимает вид:

$$T_1 = n^2(2n-1) \cdot \tau + \gamma(2n^3 + n^2)(\alpha + 64/\beta)$$

Программная реализация на C++

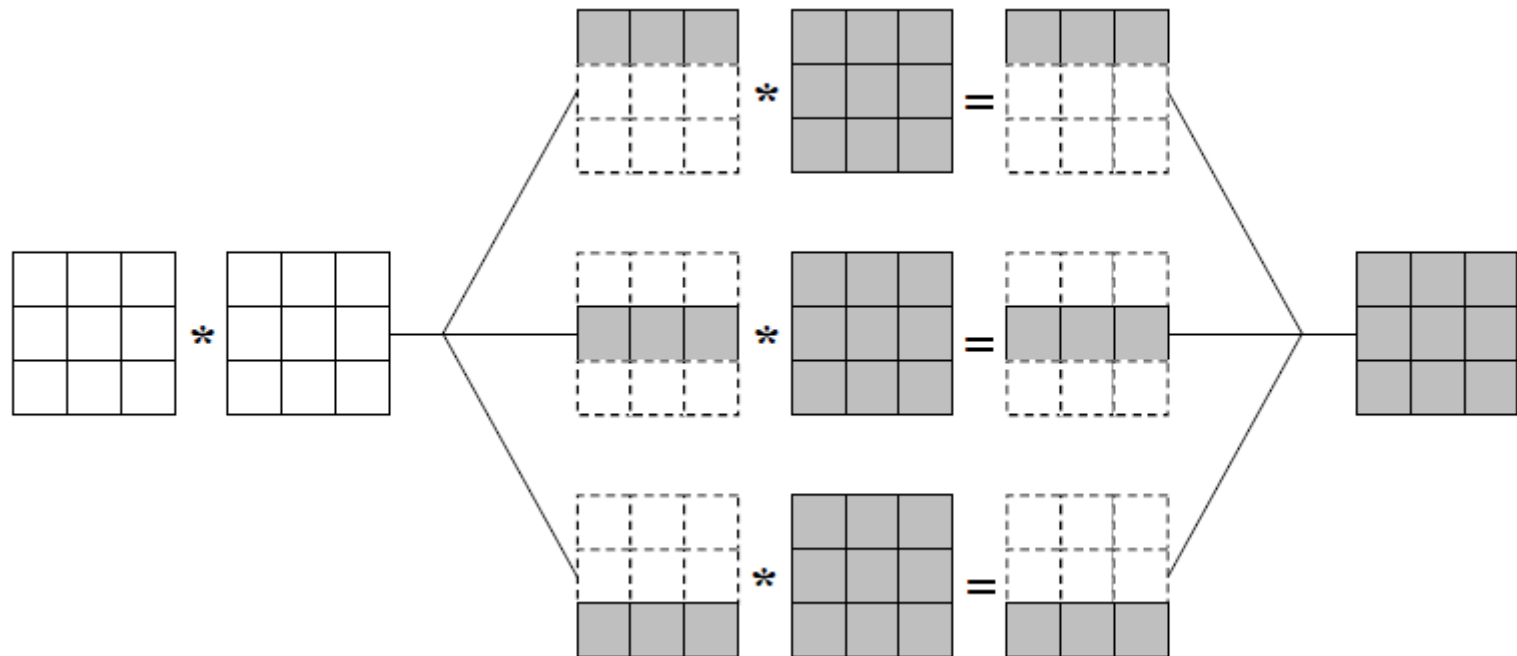
```
pAMatrix = new double [Size*Size];  
pBMatrix = new double [Size*Size];  
pCMatrix = new double [Size*Size];  
for (i=0; i<Size; i++)  
    for (j=0; j<Size; j++)  
        pCMatrix[i*Size+j] = 0;
```

```
void SerialResultCalculation(double* pAMatrix, double* pBMatrix,  
    double* pCMatrix, int Size) {  
    int i, j, k; // Loop variables  
    for (i=0; i<Size; i++)  
        for (j=0; j<Size; j++)  
            for (k=0; k<Size; k++)  
                pCMatrix[i*Size+j] += pAMatrix[i*Size+k]*pBMatrix[k*Size+j];  
}
```


Базовый параллельный алгоритм умножения матриц

- Для проведения всех необходимых вычислений каждая подзадача должна производить вычисления над элементами одной строки матрицы A и одного столбца матрицы B .
- Общее количество получаемых при таком подходе подзадач оказывается равным n^2 (по числу элементов матрицы C).

- Для вычисления одной строки матрицы C необходимо, чтобы в каждой подзадаче содержалась строка матрицы A и был обеспечен доступ ко всем столбцам матрицы B .
- Схема организации вычислений при выполнении параллельного алгоритма умножения матриц, основанного на разделении матриц по строкам:



Анализ эффективности

- Данный параллельный алгоритм обладает хорошей «локальностью вычислений». Это означает, что данные, которые обрабатывает один из потоков параллельной программы, не изменяются другим потоком.
- Нет взаимодействия между потоками, нет необходимости в синхронизации.

$$T_{calc} = (n^2 / p)(2n - 1) \cdot \tau .$$

Анализ эффективности

- Время работы с оперативной памятью

$$T_{mem} = \left(n^3 + n^3 / p + n^2 \right) (\alpha + 64 / \beta),$$

- Время выполнения параллельного алгоритма составляет

$$T_p = (n^2 / p) (2n - 1) \cdot \tau + \left(n^3 + n^3 / p + n^2 \right) (\alpha + 64 / \beta)$$

- при введении в модель показателя частоты кэш промахов:

$$T_p = (n^2 / p) (2n - 1) \cdot \tau + \gamma \left(n^3 + n^3 / p + n^2 \right) (\alpha + 64 / \beta)$$

Программная реализация (OpenMP)

```
void ParallelResultCalculation(double* pAMatrix, double* pBMatrix,  
    double* pCMatrix, int Size) {  
    int i, j, k; // Loop variables  
#pragma omp paralell for private (j, k)  
    for (i=0; i<Size; i++)  
        for (j=0; j<Size; j++)  
            for (k=0; k<Size; k++)  
                pCMatrix[i*Size+j] += pAMatrix[i*Size+k]*pBMatrix[k*Size+j];  
}
```

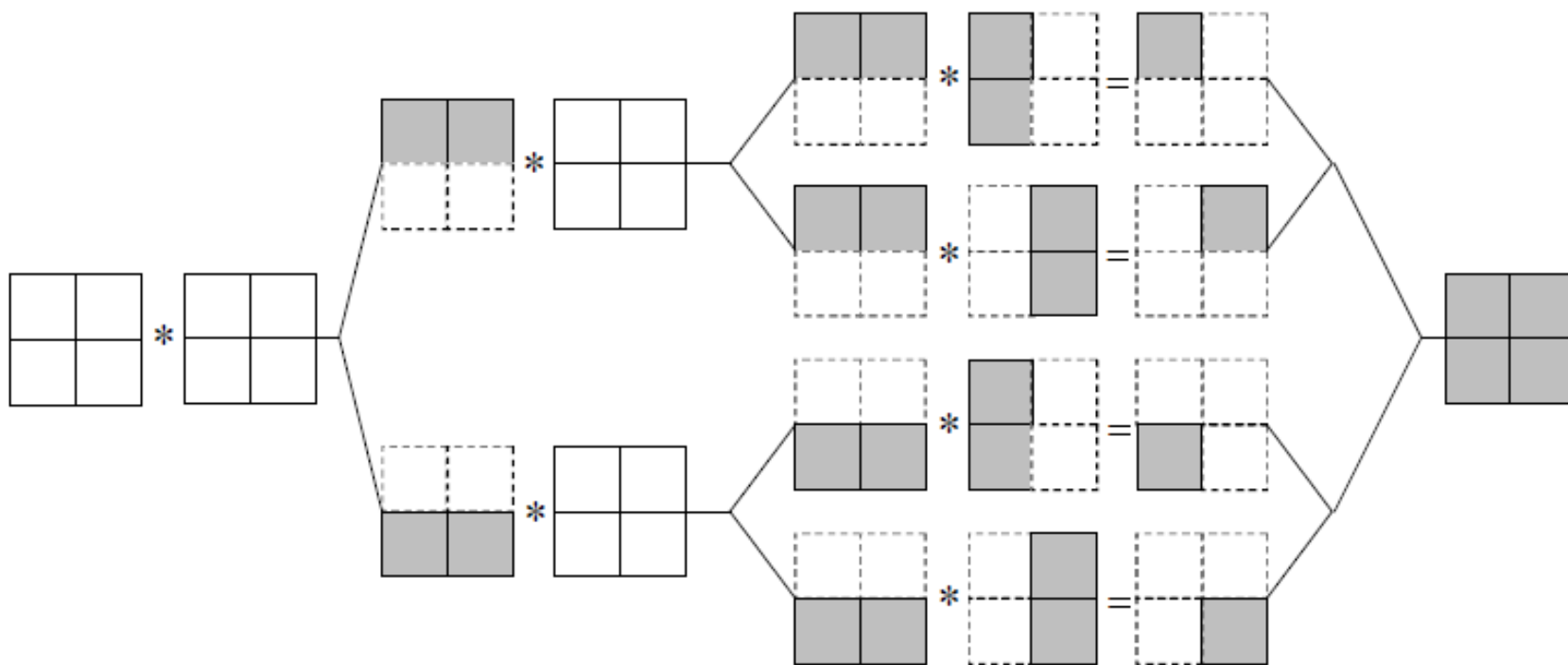
Алгоритм умножения матриц, основанный на ленточном разделении данных

- Ленточная схема разделения данных применяется и для второй перемножаемой матрицы B .
- Такой подход, в частности, позволяет улучшить локализацию данных в потоках и повысить эффективность использования кэша.

- В качестве базовой подзадачи будем рассматривать процедуру определения одного элемента результирующей матрицы C .
- Но тогда количество базовых подзадач существенно превышает число доступных вычислительных элементов.
- Определим базовую подзадачу как процедуру вычисления всех элементов прямоугольного блока матрицы C .

$$C_{i_1-i_2, j_1-j_2} = \{c_{ij} : i_1 \leq i \leq i_2, j_1 \leq j \leq j_2\}$$

- Организация параллельных вычислений при выполнении параллельного алгоритма умножения матриц, основанного на ленточном разделении данных, и использованием четырех потоков



Анализ эффективности

$$T_{calc} = (n^2 / p)(2n - 1) \cdot \tau$$

$$T_{mem} = (n^3 + n^3 / q + n^2)(\alpha + 64 / \beta)$$

$$T_p = (n^2 / p) \cdot (2n - 1)\tau + (n^3 + n^3 / q + n^2)(\alpha + 64 / \beta) + (n / q) \cdot \delta$$

$$T_p = (n^2 / p) \cdot (2n - 1)\tau + \gamma(n^3 + n^3 / q + n^2)(\alpha + 64 / \beta) + (n / q) \cdot \delta$$

Программная реализация (OpenMP)

```
void ParallelResultCalculation(double* pAMatrix, double* pBMatrix,  
    double* pCMatrix, int Size) {  
    int i, j, k;  // Loop variables  
    int NestedThreadsNum = 2;  
    omp_set_nested(true);  
    omp_set_num_threads (NestedThreadsNum);  
    #pragma omp parallel for private (j, k)  
        for (i=0; i<Size; i++)  
    #pragma omp parallel for private (k)  
        for (j=0; j<Size; j++)  
            for (k=0; k<Size; k++)  
                pCMatrix[i*Size+j] += pAMatrix[i*Size+k]*pBMatrix[k*Size+j];  
}
```

Блочный алгоритм умножения матриц

- В этом случае не только результирующая матрица, но и матрицы-аргументы матричного умножения разделяются между потоками параллельной программы на прямоугольные блоки.
- Такой подход позволяет добиться большей локализации данных и повысить эффективность использования кэш памяти.

- Исходные матрицы A , B и результирующая матрица C представляются в виде наборов блоков.

$$\begin{pmatrix} A_{00} & A_{01} & \dots & A_{0q-1} \\ \dots & \dots & \dots & \dots \\ A_{q-10} & A_{q-11} & \dots & A_{q-1q-1} \end{pmatrix} \times \begin{pmatrix} B_{00} & B_{01} & \dots & B_{0q-1} \\ \dots & \dots & \dots & \dots \\ B_{q-10} & B_{q-11} & \dots & B_{q-1q-1} \end{pmatrix} = \begin{pmatrix} C_{00} & C_{01} & \dots & C_{0q-1} \\ \dots & \dots & \dots & \dots \\ C_{q-10} & C_{q-11} & \dots & C_{q-1q-1} \end{pmatrix}$$

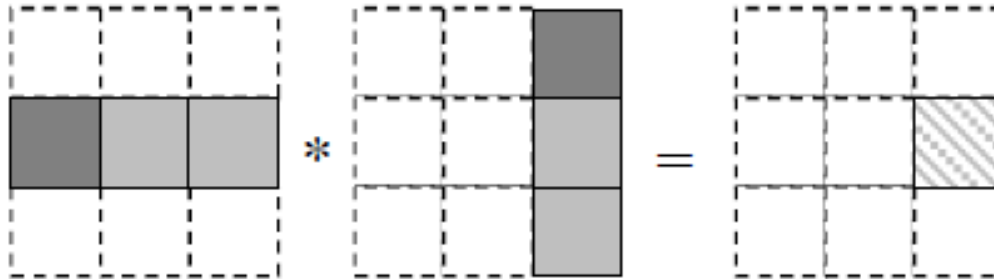
- где каждый блок C_{ij} матрицы C определяется в соответствии с выражением

$$C_{ij} = \sum_{s=0}^{q-1} A_{is} B_{sj}$$

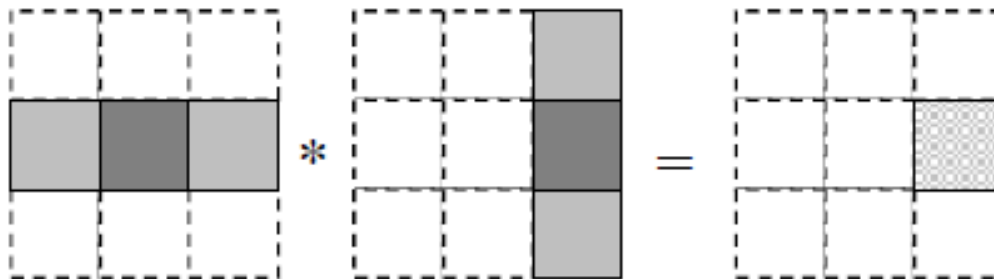
- При блочном разбиении данных для определения базовых подзадач естественным представляется взять за основу вычисления, выполняемые над матричными блоками.
- Для выполнения всех необходимых вычислений базовым подзадачам должны быть доступны соответствующие наборы строк матрицы A и столбцов матрицы B .

Схема организации блочного умножения полос

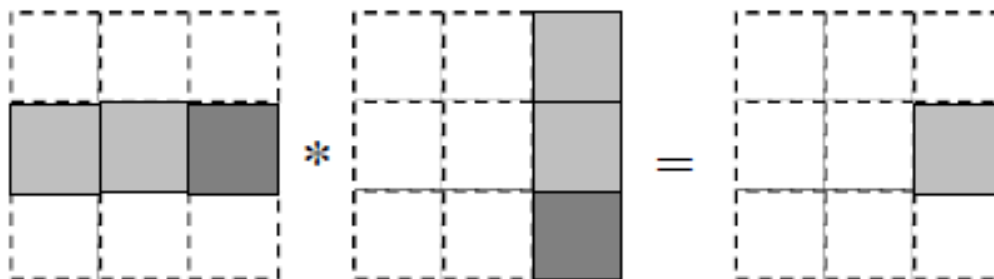
Итерация 1



Итерация 2



Итерация 3



Анализ эффективности

- Блочное разбиение полос вносит изменение лишь в порядок выполнения вычислений, общий же объем вычислительных операций при этом не изменяется.

$$T_{calc} = (n^2 / p)(2n - 1) \cdot \tau$$

- На каждой итерации алгоритма каждый поток выполняет умножение двух матричных блоков размером $(n/q) \times (n/q)$.

$$T_{mem}^1 = \left(2 \cdot (n/q)^3 + (n/q)^2 \right) (\alpha + 64 / \beta)$$

Анализ эффективности

- Общее число потоков – q^2 .
- Таким образом, время считывания данных из оперативной памяти в кэш для блочного алгоритма умножения матриц составляет:

$$T_{mem} = \left(2 \cdot (n/q)^3 + (n/q)^2 \right) (\alpha + 64/\beta) \cdot q \cdot q^2$$

- Время, необходимое на чтение данных из оперативной памяти в кэш составляет:

$$T_{mem} = \left(2 \cdot (n/q)^3 + (n/q)^2 \right) (\alpha + 64/\beta) \cdot q \cdot q = \left(2 \cdot n^3/q + n^2 \right) (\alpha + 64/\beta)$$

$$T_p = (n^2 / p) (2n - 1) \cdot \tau + \left(2 \cdot n^3/q + n^2 \right) (\alpha + 64/\beta)$$

Программная реализация (OpenMP)

```
void ParallelResultCalculation (double* pAMatrix, double* pBMatrix,  
    double* pCMatrix, int Size) {  
    int ThreadNum = 4;  
    int GridSize  = int (sqrt((double)ThreadNum));  
    int BlockSize = Size/GridSize;  
    omp set num threads(ThreadNum);  
#pragma omp parallel  
    {  
        int ThreadID = omp_get_thread_num();  
        intRowIndex = ThreadID/GridSize;  
        int ColIndex = ThreadID%GridSize;  
        for (int iter=0; iter<GridSize; iter++) {  
            for (int i=RowIndex*BlockSize; i<(RowIndex+1)*BlockSize; i++)  
                for (int j=ColIndex*BlockSize; j<(ColIndex+1)*BlockSize; j++)  
                    for (int k=iter*BlockSize; k<(iter+1)*BlockSize; k++)  
                        pCMatrix[i*Size+j] += pAMatrix[i*Size+k] * pBMatrix[k*Size+j];  
        }  
    } // pragma omp parallel  
}
```

Блочный алгоритм, эффективно использующий кэш-память

- Значительная доля времени умножения матриц тратится на многократное чтение элементов матриц A и B из оперативной памяти в кэш.
- Для организации алгоритма умножения матриц, который более эффективно использует кэш-память, воспользуемся разделением матриц на блоки.
- Разделение на блоки следует проводить таким образом, чтобы размер блока был настолько мал, что три блока, участвующие в умножении на данной итерации, возможно было поместить в кэш.

Реализация последовательного алгоритма

- Количество разбиений матриц теперь определяется не количеством потоков, а объемом кэш памяти.
- Пусть V_{cache} – объем кэш в байтах.
- Максимальный размер квадратного $k \times k$ матричного блока составляет: $k_{\max} = \lfloor \sqrt{V_{cache} / (3 \cdot 8)} \rfloor$
- Минимально необходимое число разбиений $GridSize$ при разделении матриц размером $n \times n$ составляет:

$$GridSize = \lceil n / k_{\max} \rceil.$$

- Для снижения сложности программной реализации алгоритма количество блоков по горизонтали и вертикали *GridSize* должно быть таким, чтобы размер матриц n мог быть поделен на *GridSize* без остатка.

```
void SerialResultCalculation (double* pAMatrix, double* pBMatrix,
double* pCMatrix, int Size) {
    int BlockSize = 250;
    int GridSize = int (Size/double(BlockSize));
    for (int n=0; n<GridSize; n++)
        for (int m=0; m<GridSize; m++)
            for (int iter=0; iter<GridSize; iter++)
                for (int i=n*BlockSize; i<(n+1)*BlockSize; i++)
                    for (int j=m*BlockSize; j<(m+1)*BlockSize; j++)
                        for (int k=iter*BlockSize; k<(iter+1)*BlockSize; k++)
                            pCMatrix[i*Size+j] += pAMatrix[i*Size+k] * pBMatrix[k*Size+j];
}
```

Параллельный алгоритм

- Базовая подзадача (поток) отвечает за вычисление блока результирующей матрицы.
- Однако теперь, когда количество блоков определяется не количеством потоков, а объемом кэш памяти, число блоков может существенно превосходить число доступных потоков.
- Проведем агрегацию вычислений – пусть каждый поток обеспечивает получение несколько матричных блоков результирующей матрицы C .

Анализ эффективности

- При умножении матриц с помощью рассмотренного алгоритма выполняется q^3 операций умножения матричных блоков.
- При этом, размер блоков определяется таким образом, чтобы на каждой итерации они могли быть одновременно помещены в кэш память.

$$T_{calc} = (n^2 / p)(2n - 1) \cdot \tau$$

$$T_{mem} = (n / k)^3 \cdot 3 \cdot k^2 (\alpha + 64 / \beta)$$

Анализ эффективности

- Общее время выполнения параллельного блочного алгоритма умножения матриц, ориентированного на эффективное использование кэш, может быть вычислено по формуле:

$$T_p = (n^2 / p)(2n - 1) \cdot \tau + (n / k)^3 \cdot 3 \cdot k^2 (\alpha + 64 / \beta)$$

- Если дополнительно учесть частоту кэш промахов, то выражение приобретает следующий вид:

$$T_p = (n^2 / p)(2n - 1) \cdot \tau + \gamma (n / k)^3 \cdot 3 \cdot k^2 (\alpha + 64 / \beta)$$

Программная реализация (OpenMP)

```
void ParallelResultCalculation (double* pAMatrix, double* pBMatrix,  
    double* pCMatrix, int Size) {  
    int BlockSize = 250;  
    int GridSize = int (Size/double(BlockSize));  
#pragma omp parallel for  
    for (int n=0; n<GridSize; n++)  
        for (int m=0; m<GridSize; m++)  
            for (int iter=0; iter<GridSize; iter++)  
                for (int i=n*BlockSize; i<(n+1)*BlockSize; i++)  
                    for (int j=m*BlockSize; j<(m+1)*BlockSize; j++)  
                        for (int k=iter*BlockSize; k<(iter+1)*BlockSize; k++)  
                            pCMatrix[i*Size+j] += pAMatrix[i*Size+k] * pBMatrix[k*Size+j];  
}
```


Ускорение вычислений при матричном умножении для всех четырех рассмотренных параллельных алгоритмов

